Genericitate – 3 & 4

Programare Orientată pe Obiecte

Genericitatea în subtipuri

Exemplu:

```
List<String> stringList = new ArrayList<String>();
                                        // corect!
List<Object> objectList = stringList;
                                        // operaţie corectă?
```

Presupunem că operaţia e corectă =>

- am putea introduce în *objectList* orice fel de obiect, nu doar obiecte de tip String =>
- Potenţiale erori de execuţie.
- Exemplu

```
objectList.add(new Object());
String s = objectList.get(0); // operatie ilegala!
```

=> Operaţia **nu va fi permisă de către compilator**!

Observaţie importantă

- Integer extends Object
- Integer extends Number

Dar nu:

- **Stack<Integer> extends Stack<Object>**
- **Stack<Integer> extends Stack<Number>**

Dacă *SubType* este un subtip

- (clasă descendentă sau
- subinterfaţă)

al lui *SuperType:*

Atenţie!

O structură generică:

GenericStructure <SubType>

NU este un subtip al lui:

GenericStructure < SuperType > !

Tip – Subtip in genericitate - concluzie

- Fie două tipuri de date A și B (de exemplu, *Number* și *Integer*)

- Indiferent dacă A si B sunt în relație de moștenire sau nu, *MyClass<A>* nu este în nici o relație cu *MyClass<B>!*

- Părintele comun al claselor *MyClass<A>* și *MyClass<B>* este *Object*!

“This is a common misunderstanding when it comes to programming with generics,

but it is an important concept to learn”

Wildcards

- Utilizate atunci când dorim să intrebuinţăm o structură generică drept *parametru* într-o funcţie şi nu dorim să limităm tipul de date din colecţia respectivă.
- Exemplu fără wildcard:

```
void printCollection(Collection<Object> c) {
    for (Object e : c) System.out.println(e);
}
```

- ne restricţionează să folosim la apelul funcţiei doar o colecţie cu elemente de tip *Object* (care ***nu*** *poate fi convertită la o colecţie de un alt tip*)!

Wildcards

- Această restricţie este eliminată de folosirea **wildcard**-urilor:
- Exemplu:

```
void printCollection(Collection<?> c) {
    for (Object e : c) System.out.println(e);
}
```

- Limitare: **nu putem adăuga elemente arbitrare** într-o colecţie cu wildcard-uri:

```
Collection<?> c = new ArrayList<String>();
                        // Operatie permisa
c.add(new Object());
                        // Eroare la compilare
```

- De ce?

Wildcards - observatii

- Nu putem adăuga într-o colecţie generică decât elemente **de un anumit tip**, iar wildcard-ul **nu indică un tip anume!**
- Putem adăuga elemente de tip *String?*

```
List<?> myList = new ArrayList<String>();
myList.add("Some String");
    // Eroare compilare!!
```

- Singurul element care poate fi adăugat este *null*, întrucât acesta este membru al oricărui tip referinţă!

- Rezolvare:

```
List<?> someList = new ArrayList<String>();
((ArrayList<String>)someList).add("Some String");
```

- Operaţiile de tip *getter* sunt posibile
  - rezultatul acestora poate fi mereu interpretat drept *Object*:

```
Object item = someList.get(0);
```

Exemple

- Corect? Dacă da, ce afişează?

```
List<?> someList = new ArrayList<String>();
((ArrayList<Integer>)someList).add(1);
System.out.print(someList);
```

- Corect? Dacă da, ce afişează?

```
List<?> someList = new ArrayList<String>();
((ArrayList<String>)someList).add("Some String");
((ArrayList<Integer>)someList).add(1);
Object item = someList.get(0);
System.out.println(item);
item = someList.get(1);
System.out.println(item);
```

Some String

1

Bounded Wildcards

- un *wildcard* poate fi înlocuit cu orice tip => poate deveni un inconvenient!

Mecanismul bazat pe **Bounded Wildcards:**

- permite introducerea unor restricţii asupra tipurilor ce pot inlocui un wildcard
- le obligă să se afle într-o relaţie ierarhică (de moştenire) faţă de un tip fix specificat.

*Clasă(sau interfaţă) <? **extends** Bază>*

- impune ca tipul elementelor clasei (interfeţei) să fie de tip **Bază** sau un **subtip** al acesteia!

*Clasă(sau interfaţă) <? **super** Bază>*

- impune ca tipul elementelor listei să fie de tip **Bază** sau o **superclasă** a acesteia

Wildcards - exemple

```
public void printList(List<?> list) {
      for (Object elem: list)
         System.out.print(elem + " ");
}
```

BOUNDED

- Delimitate superior

```
public double sumOfList(List<? extends Number> list) {
      double s = 0.0;
      for (Number n : list)
         s += n.doubleValue();
      return s;
}
```

- Delimitate inferior

```
public void addNumbers(List<? super Integer> list) {
      for (int i = 1; i <= 10; i++) {
            list.add(i);
      }
}
```

Exemplu

```
class User{
    protected String name = "User";
    public String getName() {
        return name;
    }
}
class Admin extends User {
    public Admin () {
        name = “Admin ";
    }
}
class Student extends User {
    public Student () {
        name = " Student ";
    }
}
```

Exemplu

```
class MyApplication {
    // bounded wildcards
    public static void listUser(List<? extends User> userList) {
        for(User item : userList)
            System.out.println(item.getName());
    }
   public static void main(String[] args) {
        List<User> pList = new ArrayList<User>();
        pList.add(new Admin());
        pList.add(new Student());
        pList.add(new User());
        MyApplication.listUser(pList);
        // Se va afisa: “Admin", “Student", "User“

       List<Admin> pList2 = new ArrayList<>();
       MyApplication.listUser(pList2);
// nu puteam face daca   listUser(List<User>…)
    }
}
```

Exemple

class D <T extends A & B & C> { /* ... */ }
-prima este clasa, daca exista o clasa

```
public class Node <T extends Number> {
	private T t;
	public void set(T t) { this.t = t; }
	public T get() { return t; }
	// Metoda generica
	public <U extends Integer> void inspect(U u){
		System.out.println("T: " + t.getClass().getName());
		System.out.println("U: " + u.getClass().getName());
	}
	public static void main(String[] args) {
		Node<Double> node = new Node<>();
		node.set(12.34); //OK
		node.inspect(1234); //OK
		node.inspect(12.34); //compile error!
		node.inspect("some text"); //compile error!
	}
}
```

Metode *generice*

= Metode care introduc un parametru de tip pentru a facilita prelucrarea unor structuri generice (date ca parametru), altul decât cei ai clasei din care fac parte.

Exemplu:

```
public class Util {
      public static <T> int countNullValues(T[] anArray) {
            int count = 0;
            for (T e : anArray)
                  if (e == null) {
                        ++count;
                  }
            return count;
      }
}

Util.countNullValues(new String[]{"a", null, "b"});
Util.countNullValues(new Integer[]{1, 2, null, 3, null});
```

Metode generice - exemple

- Exemple de implementare ale unei metode ce copiază elementele unui vector intrinsec într-o colecţie:
- Metodă corectă

```
static <T> void correctCopy(T[] a, Collection<T> c){
    for (T o : a) c.add(o); // Operatia va fi permisa
}
```

- Metodă incorectă

```
static void incorrectCopy(Object[] a, Collection<?> c){
    for (Object o : a) c.add(o);
/* Operatie incorecta, eroare la compilare: adăugarea
   elementelor într-o colecţie generică cu tip
   specificat*/
}
```

Metode generice

- putem folosi *wildcards* sau *bounded wildcards*.

- Exemple corecte:
- metodă ce copiaza elementele dintr-o listă în altă listă

```
public static <T> void copy (List<T> dest,
                             List<? extends T> src) { …}
```

- metodă de adăugare a unor elemente într-o colecţie, cu restricţionarea tipului generic

```
public <E> boolean addAll (int index, Collection<?
    extends E> c)
```

Metode generice si parametri de tip Bounded - exemplu

```
public static <T> int countGreaterThan(T[] anArray, T elem) {
    int count = 0;
    for (T e : anArray)
        if (e > elem) // compiler error
            ++count;
    return count;
}
```

- Operatorul > se aplica doar pentru tipurile primitive (short, int, double, long, float, byte, char), nu pentru a compara obiecte.
- Pentru a rezolva problema se foloseste un parametru de tip marginit de interfata Comparable<T>:

```
public interface Comparable<T> {
    public int compareTo(T o); }

public static <T extends Comparable<T>> int countGreaterThan
    (T[] anArray, T elem) {
    int count = 0;
    for (T e : anArray)
        if (e.compareTo(elem) > 0) ++count;
    return count;
}
```

Stergerea tipurilor - Type Erasure

- Genericitatea a fost introdusa in Java pentru a realiza un control al tipurilor la compilare si pentru a furniza suportul pentru programarea generica.
- Pentru a implementa genericitatea compilatorul Java aplica stergerea tipului astfel:
- Inlocuieste toti parametrii de tip din tipurile generice cu limitele lor sau cu Object daca parametrii de tip sunt nemarginiti
- Bytecod-ul generat astfel, contine doar clase, interfete si metode obisnuite
- Insereaza conversii de tip daca sunt necesare pentru a asigura type safety.
- Genereaza bridge methods pentru a asigura polimorfismul in tipurile generice extinse.
- Type erasure asigura faptul ca nicio noua clasa nu va fi creata pentru tipurile parametrizate; ca urmare, genericitatea nu genereaza runtime overhead.

Type Erasure – exemplul 1

```
public class Node<T> {
    private T data;
    private Node<T> next;
    public Node(T data, Node<T> next){
        this.data = data;
        this.next = next;
    }
    public T getData() { return data; }
        // ...
}
```

- Rezultatul dupa type erasure – inlocuire cu Object:

```
public class Node {
    private Object data;
    private Node next;
    public Node(Object data, Node next) {
        this.data = data; this.next = next; }
    public Object getData() { return data; }
    // ...
}
```

Type Erasure – exemplul 2

```
public class Node<T extends Comparable<T>> {
    private T data;
    private Node<T> next;
    public Node(T data, Node<T> next) {
        this.data = data; this.next = next; }
    public T getData() { return data; }
    // ...
}
```

- Compilatorul inlocuieste parametrul de tip T marginit cu prima clasa care-l margineste - Comparable:

```
public class Node {
    private Comparable data;
    private Node next;
    public Node(Comparable data, Node next) {
        this.data = data; this.next = next; }
    public Comparable getData() { return data; }
    // ...
}
```

Restrictii in genericitate

NU SE POT:

- Instantia tipuri generice folosind tipuri primitive
- Crea instante pentru un parametru de tip
- Declara campuri statice al caror tip este un parametru de tip
- Utiliza *cast* sau *instanceof* cu un tip parametrizat
- Crea *arrays* de un tip parametrizat
- Crea, prinde, arunca obiecte de tip parametrizat
- Supraincarca o metoda cu parametri de tip care au acelasi raw type

Instantiere tipuri generice folosind tipuri primitive - NU

```
class Pair<K, V> {
    private K key;
    private V value;
    public Pair(K key, V value) {
        this.key = key;
        this.value = value; }
    // ...
}
```

- Pair<**int, char**> p = new Pair<>(8, 'a');
  // compile-time error
- Pair<**Integer, Character**> p = new Pair<>(8, 'a');
  Se face autoboxing 8 la Integer.valueOf(8) si 'a' la Character('a'):
  Pair<Integer, Character> p = new Pair<>(Integer.valueOf(8), new Character('a'));

Creare instante pentru un parametru de tip - NU

- compile-time error:

```
public static <E> void append(List<E> list) {
    E elem;
     e = new E(); // compile-time error
    list.add(elem);
}
```

- Se poate crea un obiect de tip generic prin reflexie:

```
public static <E> void append(List<E> list,
    Class<E> cls) throws Exception {
     E elem = cls.newInstance(); // OK
     list.add(elem);
}
```

- Apel metoda:

```
List<String> ls = new ArrayList<>();
append(ls, String.class);
```

Declarare campuri statice ce au ca tip un parametru de tip - NU

```
public class MobileDevice<T> {
    private static T os; // nu
    // ...
}
```

- Daca ar fi permis, urmatorul cod ar fi confuz:

```
MobileDevice<Smartphone> phone = new
    MobileDevice<>();
MobileDevice<Pager> pager = new
    MobileDevice<>();
MobileDevice<TabletPC> pc = new
    MobileDevice<>();
```

- Deoarece campul static *os* este comun atat pentru phone, pager, cat si pc, care ar fi de fapt tipul lui *os*? Nu poate fi Smartphone, Pager, si TabletPC in acelasi timp.

Utilizare cast sau instanceof cu un tip parametrizat - NU

- Datorita stergerii tuturor parametrilor de tip din codul generic, nu se poate verifica ce tip parametrizat este folosit la executie pentru un tip generic:

```
public static <E> void test (List<E> list) {
    if (list instanceof ArrayList<Integer>)
      { // compile-time error // ... }
}
```

- Multimea tipurilor parametrizate pe care le poate primi metoda *test*:

  S = { ArrayList<Integer>, ArrayList<String> LinkedList<Character>, ... }

- La executie nu se poate face diferenta intre un ArrayList<Integer> si un ArrayList<String>. Ce putem face este sa folosim un wildcard pentru a verifica ca *list* este un ArrayList:

```
public static void test (List<?> list) {
      if (list instanceof ArrayList<?>) {
            // OK; instanceof requires a reifiable type
      }
}
```

Utilizare cast tip parametrizat - NU

- In mod normal, nu se poate converti un tip parametrizat decat daca el este parametrizat printr-un wildcard.

- Exemplu:

  List<Integer> li = new ArrayList<>();

  List<Number> ln = (List<Number>) li; // compile-time error

- Totusi, in unele cazuri compilatorul stie ca un parametru de tip este intotdeauna valid si permite conversia.

- Exemplu:

  List<String> l1 = ...;

  ArrayList<String> l2 = (ArrayList<String>)l1; // OK

Creare *vectori intrinseci* de un tip parametrizat - NU

```
List<Integer>[] arrayOfLists = new List<Integer>[2];
// compile-time error
```

- Inserare tipuri diferite intr-un vector intrinsec:

```
Object[] strings = new String[2];
strings[0] = "hi"; // OK
strings[1] = 100; //  ArrayStoreException
```

- Daca am incerca acelasi lucru cu o lista generica:

```
Object[] stringLists = new List<String>[];
    // compiler error, dar sa presupunem ca ar fi corect
stringLists[0] = new ArrayList<String>(); // OK
stringLists[1] = new ArrayList<Integer>();
/* Ar trebui sa apara ArrayStoreException, dar nu
    poate fi detectata!! */
```

Creare, prindere, aruncare obiecte de tip parametrizat - NU

- O clasa generica nu poate extinde direct sau indirect clasa Throwable:

```
// Mostenire indirecta Throwable
class MathException<T> extends Exception { /* ... */ }
        // compile-time error
// Mostenire directa Throwable
class QueueFullException<T> extends Throwable { /* ...*/ }
        // compile-time error
```

- O metoda nu poate prinde o instanta a unui parametru de tip:

```
public static <T extends Exception, J> void execute(List<J>
    jobs){
    try {
        for (J job : jobs) // ...
     } catch (T e) { // compile-time error // ... }
}
```

- Totusi, se poate folosi un parametru de tip in clauza throws:

```
class Parser<T extends Exception> {
    public void parse(File file) throws T { // OK // ... }
}
```

Supraincarcare metoda cu parametri de tip care au acelasi raw type - NU

- O clasa nu poate avea doua metode supraincarcate care vor avea aceeasi signatura – antet – dupa stergerea tipurilor (type erasure)

```
public class Example {
    public void print(Set<String> strSet) { }
    public void print(Set<Integer> intSet) { }
}
```

- compile-time error.

Aplicatii

Intrebări și exerciții

- *The Java Tutorials*, Generics

http://docs.oracle.com/javase/tutorial/java/generics/QandE/generics-questions.html

Exemplu lant apeluri

Programare Orientată pe Obiecte

Apelul în lanţ (implicit) al constructorilor

```
class Demo{
   int  value1;
   int  value2;

   Demo(){
      value1 = 1;
      value2 = 2;
      System.out.println("Inside 1st Parent Constructor");
   }
   Demo ( int a ) {
      value1 = a;
      System.out.println("Inside 2nd Parent Constructor");
   }
  public void display(){
     System.out.println("Value1 === "+value1);
     System.out.println("Value2 === "+value2);
  }
}
```

Lanţul de apeluri al constructorilor

```
class DemoChild extends Demo{
    int value3;
    int value4;
    DemoChild(){
          super(5);
           value3 = 3;
           value4 = 4;
          System.out.println("Inside the 1st Constructor of Child");
    }
    DemoChild (int a){
           this();
           value3 = a;
           System.out.println("Inside the 2nd Constructor of Child");
    }

    public void display() {…}

    public static void main(String args[]){
            DemoChild d2 = new DemoChild (11);
            d2.display();
      }
}
```

Lanţul de apeluri al constructorilor

Inside 1st Parent Constructor
Inside the 2nd Constructor of Child
Value1 === 1
Value2 === 2
Value3 === 11
Value4 === 0

// decomentare *this*
Inside 1st Parent Constructor
Inside the 1st Constructor of Child
Inside the 2nd Constructor of Child
Value1 === 1
Value2 === 2
Value3 === 11
Value4 === 4

// decomentare şi *super ???*

Lanţul de apeluri al constructorilor

Inside 1st Parent Constructor
Inside the 2nd Constructor of Child
Value1 === 1
Value2 === 2
Value3 === 11
Value4 === 0

// decomentare *this*
Inside 1st Parent Constructor
Inside the 1st Constructor of Child
Inside the 2nd Constructor of Child
Value1 === 1
Value2 === 2
Value3 === 11
Value4 === 4

// decomentare şi *super*
Inside 2nd Parent Constructor
Inside the 1st Constructor of Child
Inside the 2nd Constructor of Child
Value1 === 5
Value2 === 0
Value3 === 11
Value4 === 4

Legare statică/dinamică – static/dynamic binding în Java

Programare Orientată pe Obiecte

Ce va afişa următorul program?

```
class TestEgal{
    public boolean equals ( TestEgal other ) {
        System.out.println( "In equals din TestEgal" );   return false;
    }
// public boolean equals ( Object other ) – mostenita de la Object
public static void main( String [] args ) {
        Object t1 = new TestEgal(), t2 = new TestEgal();
        TestEgal t3 = new TestEgal();
        Object o1 = new Object();
        int count = 0;
        System.out.println( count++ ); // afiseaza 0
        t1.equals( t2 ) ;
        System.out.println( count++ ); // afiseaza 1
        t1.equals( t3 );
        System.out.println( count++ ); // afiseaza 2
        t3.equals( o1 );
        System.out.println( count++ ); // afiseaza 3
        t3.equals(t3);
        System.out.println( count++ ); // afiseaza 4
        t3.equals(t2);
    }
}
```

Rezolvare

Output:

0

1

2

3

In equals din TestEgal

4

Atenţie! Este vorba de overload nu override!

Metoda *public boolean equals (TestEgal other)*

Nu supradefineşte metoda *public boolean equals (Object other)* din clasa Object, ci o supraîncarcă!

Deci, static binding! Singurul loc în care se apleează equals din TestEgal este **t3.equals(t3);** !

Exerciţiu propus

```
class Ana {
        public void print(Ana p) {
           System.out.println("Ana 1\n");
        }
}
class Mihai extends Ana {
        public void print(Ana p) {
           System.out.println("Mihai 1\n");
        }
        public void print(Mihai l) {
           System.out.println("Mihai 2\n");
        }
}
class Dana extends Mihai {
        public void print(Ana p) {
           System.out.println("Dana 1\n");
        }
        public void print(Mihai l) {
           System.out.println("Dana 2\n");
        }
        public void print(Dana b) {
           System.out.println("Dana 3\n");
        }
}
```

Exerciţiu propus

```
public class Test{
    public static void main (String [] args) {
       Mihai stud1 = new Dana();
        Ana stud2 = new Mihai();
        Ana stud3 = new Dana();
        Dana stud4 = new Dana();
        Mihai stud5 = new Mihai();
 1  stud1.print(new Ana()); // Dana 1
 2  ((Dana)stud1).print(new Mihai()); // Dana2
 3  ((Mihai)stud2).print(new Ana()); // Mihai 1
 4  stud2.print(new Dana()); // Mihai 1
 5  stud2.print(new Mihai()); // Mihai 1
 6  stud3.print(new Dana()); // Dana 1
 7  stud3.print(new Ana()); // Dana 1
 8  stud3.print(new Mihai()); // Dana 1
 9  ((Dana)stud3).print(new Mihai()); // Dana 2
10  ((Dana)stud3).print(new Dana()); // Dana 3
11  stud4.print(new Dana()); // Dana 3
12  stud4.print(new Ana()); // Dana 1
13  stud4.print(new Mihai()); // Dana 2
14  stud5.print(new Dana()); // Mihai 2
15  stud5.print(new Mihai()); // Mihai 2
16  stud5.print(new Ana());  // Mihai 1
17  ((Ana)stud5).print(new Mihai()); // Mihai 1
} }
```

Ierarhie

Ana – print (Ana)

|

Mihai – print (Ana), print (Mihai)

|

Dana – print (Ana), print (Mihai), print (Dana)

Tip – nume -> obiect

- **Mihai - stud1 -> Dana**
- **Ana - stud2 -> Mihai**
- **Ana - stud3 -> Dana**
- **Dana - stud4 -> Dana**
- **Mihai - stud5 -> Mihai**